

Recap Checkpoint 5 — Cumulative: Component 1 (1.1, 1.3, 1.4) + Component 2 (2.1–2.3, incl. Algorithms) — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Mark scheme

Q1 [3] (AO1). - Difference (1): Von Neumann uses a **single shared memory and bus** for both instructions and data; Harvard uses **separate memories/buses** for instructions and data. - Advantage of Von Neumann (1): **simpler/cheaper to build**, flexible memory use. - Advantage of Harvard (1): instructions and data can be **fetches simultaneously** → faster; common in embedded/DSP systems.

Examiner tip: the core distinction is shared vs separate storage for instructions and data.

Q2 [4] (AO2). - (a) 3B: $3 \times 16 = 48$, $B = 11 \rightarrow 48 + 11 = 59$ (1 method, 1 answer). - (b) Bits = $64 \times 48 \times 4 = 12\,288$ bits; $\div 8 = 1\,536$ bytes; $\div 1\,024 = 1.5$ KiB (1 for bits/bytes working, 1 for 1.5 KiB).

Examiner tip: image size = width $\times$ height $\times$ colour depth (bits). Convert bits $\rightarrow$ bytes ($\div 8$) $\rightarrow$ KiB ($\div 1024$).

Q3 [4] (AO2). - (a) $A \cdot B + A \cdot \bar{B} = A \cdot (B + \bar{B}) = A \cdot 1 = A$ (1 for answer **A**); laws: **Distributive** then **complement/inverse** ($B + \bar{B} = 1$) and **identity** (1 for naming a relevant law). - (b) Truth table for $Q = (A \text{ OR } B) \text{ AND NOT } B$:

A	B	Q
0	0	0
0	1	0
1	0	1
1	1	0

- 1 mark for the two **B=1** rows both 0; 1 mark for **A=1, B=0** $\rightarrow$ 1 and **A=0, B=0** $\rightarrow$ 0.

Examiner tip: NOT B forces $Q = 0$ whenever $B = 1$, regardless of A.

Q4 [3] (AO1). - (a) A foreign key is an attribute in one table that is the **primary key of another table**, used to link the two tables (1). - (b) 3NF removes **repeating/redundant data** by ensuring non-key attributes depend **only on the (whole) primary key** and not on other non-key attributes (transitive dependencies removed) (1), so each fact is stored **once**, reducing redundancy and update anomalies (1).

Examiner tip: link 3NF to "no non-key attribute depends on another non-key attribute".

Q5 [4] (AO2/AO1). - (a) Procedural organises code as **sequences of procedures/functions acting on data**; OOP organises code as **objects bundling data + methods** (1). - (b) Encapsulation = **bundling attributes and the methods that operate on them inside an object/class**, with attributes typically

private and accessed via methods (1); benefit: **protects data integrity / hides internal detail**, so internals can change without breaking other code (1). - (c) Inheritance = a class (**subclass/child**) **acquires the attributes and methods of another class** (superclass/parent), allowing reuse (1).

Examiner tip: encapsulation = data hiding; benefit must be a consequence (e.g. controlled access, maintainability).

Q6 [4] (AO2). Merge sort on [8, 3, 5, 1]: - Splits: [8, 3, 5, 1] → [8, 3] / [5, 1] → [8], [3], [5], [1] (1). - Merge: [8], [3] → [3, 8]; [5], [1] → [1, 5] (1). - Merge: [3, 8] + [1, 5] → [1, 3, 5, 8] ✓ (1). - Worst-case time complexity: **$O(n \log n)$** (1).

Examiner tip: merge sort always splits to single elements then merges in sorted order — $O(n \log n)$ in all cases.

Q7 [4] (AO1/AO2). - (a) Linear search **$O(n)$** (1); binary search **$O(\log n)$** (1); bubble sort **$O(n^2)$** (1). - (b) Loop runs n times × binary search $O(\log n)$ each → **$O(n \log n)$** (1).

Examiner tip: combine nested costs by multiplying — n iterations each costing $\log n$ gives $n \log n$.

Q8 [4] (AO2). - In-order (Left, Node, Right): **10, 20, 30, 40, 55, 70** (1). - Property: the in-order traversal of a BST is in **ascending sorted order** (1). - Inserting 27: compare with **40** ($27 < 40 \rightarrow$ go left) (1); compare with **20** ($27 > 20 \rightarrow$ go right); compare with **30** ($27 < 30 \rightarrow$ go left); inserted as 30's left child — comparison order **40, 20, 30** (1).

Examiner tip: traverse from the root, going left when smaller and right when larger, until an empty position is reached.

Q9 [5] (AO3). Indicative content (up to 5): - Linear search is **$O(n)$** — checks items one by one, so time grows linearly; with millions of songs this is slow (1). - Binary search is **$O(\log n)$** — halves the search space each comparison, so even millions of items take only ~20–21 comparisons (1–2). - Binary search **requires the data to be sorted** (precondition), which is why the library is kept sorted by title (1). - Trade-off (1): keeping the library sorted makes **insertions/updates more costly** (must maintain order), so adding new songs is slower; extra effort/structure (e.g. index) is needed. - Reward reasoning that explicitly compares $O(n)$ vs $O(\log n)$ and identifies a sorting cost.

Examiner tip: a top answer quantifies the gain ($\log_2$ of millions ≈ 20 comparisons) and names the cost of maintaining sorted order.

Total: 35 marks.